JavaScript Array Sort

Previous Next

Alphabetic Sort

Array sort()
Array reverse()
Array toSorted()
Array toReversed()
Sorting Objects

Numeric Sort

Numeric Sort
Random Sort
Math.min()
Math.max()
Home made Min()
Home made Max()

See Also:

Array Tutorial

Array Basic Methods

Array Search Methods

Array Iteration Methods

Array Reference

Sorting an Array

The `sort()` method sorts an array alphabetically:

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
fruits.sort();
```

Try it Yourself »

Reversing an Array

The `reverse()` method reverses the elements in an array:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
fruits.reverse();
```

Try it Yourself »

By combining `sort()` and `reverse()`, you can sort an array in descending order:

Example

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
fruits.sort();
fruits.reverse();
```

Try it Yourself »

JavaScript Array toSorted() Method

ES2023 added the `toSorted()` method as a safe way to sort an array without altering the original array.

The difference between `toSorted()` and `sort()` is that the first method creates a new array, keeping the original array unchanged, while the last method alters the original array.

```
const months = ["Jan", "Feb", "Mar", "Apr"];
const sorted = months.toSorted();
```

Try it Yourself »

JavaScript Array toReversed() Method

ES2023 added the `toReversed()` method as a safe way to reverse an array without altering the original array.

The difference between `toReversed()` and `reverse()` is that the first method creates a new array, keeping the original array unchanged, while the last method alters the original array.

Example

```
const months = ["Jan", "Feb", "Mar", "Apr"];
const reversed = months.toReversed();
```

Try it Yourself »

Numeric Sort

By default, the `sort()` function sorts values as **strings**.

This works well for strings ("Apple" comes before "Banana").

If numbers are sorted as strings, "25" is bigger than "100", because "2" is bigger than "1".

Because of this, the `sort()` method will produce incorrect result when sorting numbers.

You can fix this by providing a **compare function**:

Example

```
const points = [40, 100, 1, 5, 25, 10];
points.sort(function(a, b){return a - b});
```

Use the same trick to sort an array descending:

Example

```
const points = [40, 100, 1, 5, 25, 10];
points.sort(function(a, b){return b - a});
```

Try it Yourself »

ADVERTISEMENT

The Compare Function

The purpose of the compare function is to define an alternative sort order.

The compare function should return a negative, zero, or positive value, depending on the arguments:

```
function(a, b){return a - b}
```

When the `sort()` function compares two values, it sends the values to the compare function, and sorts the values according to the returned (negative, zero, positive) value.

If the result is negative, `a` is sorted before `b`.

If the result is positive, `b` is sorted before `a`.

If the result is 0, no changes are done with the sort order of the two values.

When comparing 40 and 100, the `sort()` method calls the compare function(40, 100).

The function calculates 40 - 100 `(a - b)`, and since the result is negative (-60), the sort function will sort 40 as a value lower than 100.

You can use this code snippet to experiment with numerically and alphabetically sorting:

```
<button onclick="myFunction1()">Sort Alphabetically</button>
<button onclick="myFunction2()">Sort Numerically</button>

<p id="demo"></p>

<script>
const points = [40, 100, 1, 5, 25, 10];
document.getElementById("demo").innerHTML = points;

function myFunction1() {
  points.sort();
  document.getElementById("demo").innerHTML = points;
}

function myFunction2() {
  points.sort(function(a, b){return a - b});
  document.getElementById("demo").innerHTML = points;
}
</script>
```

Try it Yourself »

Sorting an Array in Random Order

Using a sort function, like explained above, you can sort an numeric array in random order

Example

```
const points = [40, 100, 1, 5, 25, 10];
points.sort(function(){return 0.5 - Math.random()});
```

Try it Yourself »

The points.sort() method in the example above is not accurate. It will favor some numbers over others.

The most popular correct method, is called the Fisher Yates shuffle, and was introduced in data science as early as 1938!

In JavaScript the method can be translated to this:

Example

```
const points = [40, 100, 1, 5, 25, 10];

for (let i = points.length -1; i > 0; i--) {
  let j = Math.floor(Math.random() * (i+1));
  let k = points[i];
  points[i] = points[j];
  points[j] = k;
}
```

Try it Yourself »

Find the Lowest (or Highest) Array Value

There are no built-in functions for finding the max or min value in an array.

To find the lowest or highest value you have 3 options:

- Sort the array and read the first or last element
- Use Math.min() or Math.max()
- Write a home made function

Find Min or Max with sort()

After you have sorted an array, you can use the index to obtain the highest and lowest values.

Sort Ascending:

Example

```
// and points[points.length-1] contains the highest value
```

Try it Yourself »

Sort Descending:

Example

```
const points = [40, 100, 1, 5, 25, 10];
points.sort(function(a, b){return b - a});
// now points[0] contains the highest value
// and points[points.length-1] contains the lowest value
```

Try it Yourself »

Note

Sorting a whole array is a very inefficient method if you only want to find the highest (or lowest) value.

Using Math.min() on an Array

You can use `Math.min.apply` to find the lowest number in an array:

Example

```
function myArrayMin(arr) {
  return Math.min.apply(null, arr);
}
```

Try it Yourself »

`Math.min.apply(null, [1, 2, 3])` is equivalent to `Math.min(1, 2, 3)`.

Example

```
function myArrayMax(arr) {
  return Math.max.apply(null, arr);
}
```

Try it Yourself »

`Math.max.apply(null, [1, 2, 3])` is equivalent to `Math.max(1, 2, 3)`.

JavaScript Array Minimum Method

There is no built-in function for finding the lowest value in a JavaScript array.

The fastest code to find the lowest number is to use a **home made** method.

This function loops through an array comparing each value with the lowest value found:

Example (Find Min)

```
function myArrayMin(arr) {
  let len = arr.length;
  let min = Infinity;
  while (len--) {
    if (arr[len] < min) {
      min = arr[len];
    }
  }
  return min;
}
```

Try it Yourself »

JavaScript Array Maximum Method

There is no built-in function for finding the highest value in a JavaScript array.

Example (Find Max)

```
function myArrayMax(arr) {
  let len = arr.length;
  let max = -Infinity;
  while (len--) {
    if (arr[len] > max) {
      max = arr[len];
    }
  }
  return max;
}
```

Try it Yourself »

Sorting Object Arrays

JavaScript arrays often contain objects:

Example

```
const cars = [
  {type:"Volvo", year:2016},
  {type:"Saab", year:2001},
  {type:"BMW", year:2010}
];
```

Even if objects have properties of different data types, the `sort()` method can be used to sort the array.

The solution is to write a compare function to compare the property values:

Example

```
cars.sort(function(a, b){return a.year - b.year});
```

Comparing string properties is a little more complex:

Example

```
cars.sort(function(a, b){
  let x = a.type.toLowerCase();
  let y = b.type.toLowerCase();
  if (x < y) {return -1;}
  if (x > y) {return 1;}
  return 0;
});
```

Try it Yourself »

Stable Array sort()

ES2019 **revised** the Array `sort()` method.

Before 2019, the specification allowed unstable sorting algorithms such as QuickSort.

After ES2019, browsers must use a stable sorting algorithm:

When sorting elements on a value, the elements must keep their relative position to other elements with the same value.

Example

```
const myArr = [
  {name:"X00",price:100 },
  {name:"X01",price:100 },
  {name:"X02",price:100 },
  {name:"X03",price:100 },
  {name:"X04",price:110 },
  {name:"X05",price:110 },
  {name:"X06",price:110 },
  {name:"X07",price:110 }
];
```

Try it Yourself »

```
X01 100
X03 100
X00 100
X03 100
X05 110
X04 110
X06 110
X07 110
```

Complete JavaScript Reference

For a complete reference to all JavaScript properties and methods, with full descriptions and many examples, go to:

W3Schools' Full JavaScript Reference.

The reference inludes all JavaScript updates from 1999 to 2025.

Exercise

Consider the following code:

```
const fruits = ['Apple', 'Orange', 'Banana'];
fruits.reverse();
```

What will the result of the `fruits` array?

- ['Orange', 'Banana', 'Apple']
- ['Banana', 'Orange', 'Apple']
- ['Apple', 'Banana', 'Orange']
- ['Orange', 'Apple', 'Banana']

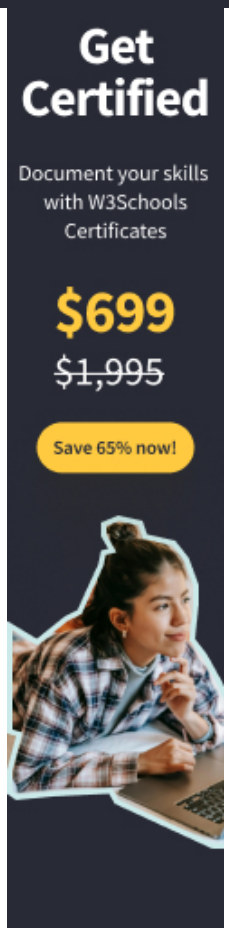
❮ Previous
Next ❯

Track your progress - it's free!
Sign Up
Log in

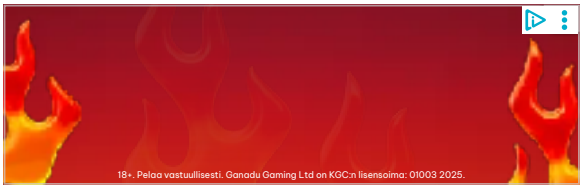
ADVERTISEMENT

COLOR PICKER

ADVERTISEMENT



ADVERTISEMENT



ADVERTISEMENT

Top Tutorials

HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial
C++ Tutorial
jQuery Tutorial

Top References

HTML Reference
CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
Angular Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

FORUM ABOUT ACADEMY

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning. Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookie and privacy policy.

Copyright 1999-2025 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.